

1. The Perceptron

A perceptron computes $y = f(w \cdot x + b)$: a weighted sum of inputs x with weights w and bias b , passed through activation function f . It is a linear classifier; the decision boundary is a hyperplane.

2. Activation Functions

Sigmoid: $s(z) = 1 / (1 + e^{-z})$. Outputs in $(0,1)$; saturates and causes vanishing gradients in deep nets.

ReLU: $f(z) = \max(0, z)$. Cheap, sparse, the modern default; can suffer from 'dying ReLU' units.

Tanh: outputs in $(-1,1)$; zero-centred, still saturates.

Softmax: converts logits into a probability distribution over classes; used in the output layer for classification.

3. Loss Functions

Mean squared error: $L = (1/n) * \sum((y_{\text{pred}} - y_{\text{true}})^2)$.

Cross-entropy loss for classification: $L = -\sum(y * \log(p))$.

The loss measures how wrong the network is; training minimises it.

4. Gradient Descent and Backpropagation

Update rule: $w \leftarrow w - lr * dL/dw$, where lr is the learning rate.

Backpropagation applies the chain rule to propagate the loss gradient from the output layer back to every weight.

Variants: stochastic gradient descent (SGD), mini-batch GD, Adam.

Learning rate too high $\rightarrow$ divergence; too low $\rightarrow$ slow convergence.

5. Training Process (standard pipeline)

Step 1: initialise weights randomly (small values, symmetry break).

Step 2: forward pass - compute predictions for a batch.

Step 3: compute the loss against the true labels.

Step 4: backward pass - backpropagate gradients.

Step 5: update weights with the optimiser.

Step 6: repeat for many epochs; monitor validation loss.

6. Overfitting and Regularisation

Overfitting: low training error, high validation error - the model memorises instead of generalising.

Remedies: more data, dropout, L2 weight decay, early stopping.

7. Self-check

Q1. Why does a network of only linear layers collapse to one layer?

Q2. Derive the derivative of the sigmoid in terms of $s(z)$.

Q3. What symptom in the validation curve signals overfitting?